ZetaShield V2.0 — The Rehearsal File

Master Presentation Script, Live Demo Runbook & Viva Defense Manual

Course: *DSN4091 Capstone Project Phase 1 · Review 2* | ***Group:*** *39* | ***Date:*** *October 2026*
Supervisor: *Dr. Sajjad Ahmed* | ***Program Chair:*** *Dr. D. Saravanan*
Team: *Polisetti Hari Sri Raj Karthik (Lead), Subeer Srivastava, Sahil Shankar Thakkar, Shaivyaa Sharma, Sameep Upadhyay*

DOCUMENT MISSION: This manual guarantees seamless coordination during the 24-minute Review 2 presentation and 6-minute live demonstration. It contains word-for-word spoken scripts for every slide, the step-by-step live terminal execution runbook, a territorial defense Q&A; bank covering 20 tough panel questions, and emergency disaster-recovery protocols. Every member must memorize their designated sections and core lines.

Part 1: The Speaker Map & Slide Allocation

Total presentation duration is strictly budgeted at **24 minutes of presentation + 6 minutes of live demonstration**, leaving buffer for committee cross-examination. Each member owns clear thematic territory to avoid stepping on each other's toes.

Slide	Topic & Slide Title	Speaker	Time	Core Theme & Deliverable
01	Title & Project Overview	**Karthik**	1.0m	Welcome, thesis statement, team introduction, project vision
02	The Problem: Flat Networks & Lateral Movement	**Karthik**	1.5m	Perimeter failure, OWASP A01, east-west attack surface, market gap
03	Consolidated Gateway Architecture	**Karthik**	2.0m	4-hop sidecar vs 2-hop consolidated gateway, containerized mesh
04	Three Checkpoints, Zero Trust	**Subeer**	2.0m	Sequential enforcement pipeline: Identity mTLS → Policy PBAC → Route/Audit
05	Trust the Issuer, Not the Claim	**Sahil**	2.0m	Cryptographic SPIFFE URI in X.509 SAN vs spoofable HTTP headers, CA chain
06	Deny-by-Default & Hot-Reload	**Subeer**	2.0m	Declarative policy.yaml, micro-segmentation logic, atomic in-memory hot-reload
07	The 7-Test Adversarial Suite	**Subeer**	2.0m	7 attack vectors: authorized, anonymous, 403, micro-seg, forged, 503 fail-closed
08	Live Demonstration Execution	**Karthik (Lead) + Subeer/Sameep**	6.0m	Live Docker launch, curl isolation proof, 7 tests run, hot-reload flex, audit log
09	Docker Network Isolation	**Karthik**	1.5m	Answering Review 1: loopback vs network namespace isolation (zeta-net bridge)
10	Sub-Millisecond Enforcement Benchmark	**Karthik**	1.5m	500 sequential requests: 0.75ms direct vs 1.35ms gateway = ~0.60ms P50 overhead
11	Engineering Rigor: 19 Unit Tests & CI	**Sameep**	1.5m	14 policy tests + 5 routing tests, table-driven Go testing, GitHub Actions CI
12	Three-Version Design Evolution	**Shaivyaa**	1.5m	V0.1 sidecar → V1.0 consolidated → V2.0 Docker mesh & hot-reload
13	Phase 2 Technical Roadmap	**Shaivyaa**	1.5m	React admin console, public VPS deployment, automated red-team container, eBPF
14	Project Outcomes & Industry Value	**Karthik**	1.0m	Identity replaces IP, provable guarantees at 0.6ms, enterprise applicability
15	Team Contributions Breakdown	**Sameep**	1.0m	Engineering lifecycle ownership across design, coding, crypto, testing, and DevOps
16	Summary & Technical Q&A; Defense	**Karthik (Lead) + All Members**	1.0m + Q&A;	Final verdict: code complete, containerized, benchmarked. Transition to panel Q&A;

Comprehensive Team Contributions Matrix

This breakdown reflects the division of engineering labor submitted in the project documentation. Each member has deep domain mastery over their allocated components.

Member & Role	Technical Focus Areas	Concrete Engineering Deliverables	Defense Territory

Karthik P.H.S.R (23BCY10308) *Project Lead*	**Core Gateway, Networking, Benchmarking, DevOps & Demo** (Heaviest Engineering Load)	• Designed & built Go L7 gateway engine (main.go, 260 LOC) • Reverse proxy forwarding & custom fail-closed ErrorHandler • Multi-stage Docker mesh (Dockerfile.gateway, Compose) • Latency benchmarking suite (cmd/loadtest, 500 requests) • GitHub Actions CI pipeline & live demo driver	System architecture, SPOF mitigation, performance latency, Docker networking, V0.1 consolidation
Subeer Srivastava (23BCY10143) *Policy Architect*	**Policy Engine, Deny-by-Default Logic & Hot-Reload** (Second-Heaviest Load)	• Designed declarative policy.yaml schema (PBAC model) • Implemented in-memory evaluation & multi-method matching • Built zero-downtime hot-reload watcher (os.Stat + ticker) • Concurrency safety with sync.RWMutex read/write locks • Engineered 7-test matrix & demo policy flex	Policy evaluation algorithm, race conditions, atomic reload, fail-closed config errors, fsnotify trade-offs
Sahil S. Thakkar (23BCY10043) *Crypto & Identity Lead*	**PKI Architecture, mTLS & SPIFFE Cryptography**	• Engineered Root CA & per-service certificate generation scripts • Implemented SPIFFE URI identity encoding in X.509 SAN extensions • Fixed proxy certificate IP/DNS SAN verification defect • Created adversarial rogue keypair for Test 6 (forged SAN) • Enforced TLS 1.2+ & RequireAndVerifyClientCert	X.509 validation, CA trust chains, SAN vs CN identity, certificate expiry/revocation, MitM resistance
Sameep Upadhyay (23BCY10187) *Verification Engineer*	**Test Automation, CI/CD Pipeline & Audit Subsystem**	• Authored 19 table-driven unit tests (policy_test.go) • Implemented 14 policy checks + 5 longest-prefix route tests • Built adversarial client harness (mock-apps/client.py) • Validated container isolation proof via host-side connection probes • Configured structured JSONL audit record schemas	Test edge-cases, regression testing, CI automation, audit log forensics, connection-refused verification
Shaivyaa Sharma (23BCY10020) *Security Analyst*	**Threat Modeling, Related Work & Phase 2 Architecture**	• Conducted systematic literature review (NIST, OWASP, Istio, Cilium) • Formulated 6-scenario adversary threat model (T1–T6) • Tracked V0.1 → V1.0 → V2.0 architectural evolution matrix • Architected Phase 2 React Admin Console & eBPF integration plan • Project report lead & academic documentation	Threat modeling, industry comparison (Istio/Envoy), Phase 2 feasibility, real-world deployment

Part 2: Word-for-Word Speaking Scripts

Memorize the exact phrasing below. Text formatted in **bold red italics** indicates critical *'Money Lines'* designed to impress panel evaluators and demonstrate security engineering maturity.

SLIDE 1: Title & Introduction (1.0 min)

SPEAKER: Polisetti Hari Sri Raj Karthik (Lead)

"Good morning respected supervisor Dr. Sajjad Ahmed, Program Chair Dr. Saravanan, and members of the evaluation committee. We are Group 39, presenting our Capstone Project Phase 1, Review 2: **ZetaShield V2.0 — Containerized Zero-Trust Micro-Segmentation Gateway**.
My name is Karthik, and I am joined by my team members: Subeer Srivastava, Sahil Thakkar, Shaivyaa Sharma, and Sameep Upadhyay.
In our first review, we presented a preliminary prototype. Today, we present a completely transformed, production-ready system: an identity-aware Layer-7 security gateway written in Go, fully containerized in Docker, with mutual TLS, declarative deny-by-default policies, live policy hot-reload, and a measured sub-millisecond enforcement overhead. Let us begin with the problem we set out to solve."

SLIDE 2: The Problem: Flat Networks & Lateral Movement (1.5 min)

SPEAKER: Polisetti Hari Sri Raj Karthik (Lead)

"Traditional cybersecurity relies heavily on perimeter firewalls operating at Layers 3 and 4. While this protects the castle gates, it creates a dangerous topology: *a hard exterior, but a completely soft, flat interior*. Once an adversary breaches the perimeter—through a phished credential, a compromised third-party package, or a remote exploit—they find themselves inside a flat internal network where every service can freely talk to every other service.
The Open Web Application Security Project (OWASP) ranks **Broken Access Control as the #1 threat in its 2025 Top 10**. Furthermore, NIST Special Publication 800-207 mandates Zero-Trust Architecture precisely to neutralize this risk.
Enterprise solutions like Istio and Envoy exist, but they demand complex Kubernetes clusters, massive resource overhead, and hundreds of thousands of lines of code. Our mission was clear: to engineer a lightweight, standalone, deployable Layer-7 enforcement engine that eliminates implicit trust between internal microservices."

MONEY LINE: "The fundamental problem we are solving is stopping attackers from moving freely once inside the perimeter."

SLIDE 3: Consolidated Gateway with Containerized Isolation (2.0 min)

SPEAKER: Polisetti Hari Sri Raj Karthik (Lead)

"To solve this without excessive latency, we underwent a major architectural evolution. In Review 1, our V0.1 prototype used a 4-hop sidecar mesh: client to client-proxy, to server-proxy, to external control plane, to backend. You rightly critiqued that design for unnecessary layering and latency.
We listened, and engineered the **V1.0 Consolidated 2-Hop Gateway**, now upgraded to **V2.0 with full Docker containerization**.
Here is the request flow: calling services present a cryptographic client certificate to the Gateway over mutual TLS on port 8443. The Gateway terminates TLS, extracts the service identity, evaluates our deny-by-default policy, routes the request to the upstream backend, and logs the complete decision to an audit trail.
Most importantly: our backend microservices—Orders and Payments—publish **no host ports**. They are physically unreachable from the host or outside network, residing purely in an internal Docker bridge network. I will now hand over to Subeer to explain our enforcement pipeline."

MONEY LINE: "Two layers. One enforcement point. All backends physically unreachable."

SLIDE 4: Three Checkpoints, Zero Trust (2.0 min)

SPEAKER: Subeer Srivastava (Policy Architect)

"Thank you, Karthik. At the core of ZetaShield is a strict, sequential enforcement pipeline comprising three mandatory checkpoints. Every single HTTP request must survive all three checkpoints before a single byte reaches a backend.
Checkpoint 1 is Identity via mTLS: Before any application data is parsed, the gateway verifies the caller's X.509 certificate against our private Root CA. If a caller presents no certificate, an expired certificate, or a forged certificate, the connection is instantly aborted at the TLS handshake level.
Checkpoint 2 is Policy-Based Access Control (PBAC): We extract the caller's identity and evaluate it against our declarative policy.yaml ruleset. Our engine is **deny-by-default**: only explicit ALLOW rules pass. If an identity has no rule for that path or HTTP method, it receives an instant HTTP 403 Forbidden.
Checkpoint 3 is Routing and Forensic Audit: Permitted requests are routed via longest-prefix matching to the internal backend. Concurrently, an immutable, append-only JSONL audit log records the exact timestamp, identity, method, path, decision, and forensic rationale.
If the backend service is offline, the gateway catches the dial error and fails closed with a controlled HTTP 503. Now, Sahil will explain how we make identity unforgeable."

MONEY LINE: "Failure is never permission. When a check fails, the request dies right there."

SLIDE 5: Trust the Issuer, Not the Claim (2.0 min)

SPEAKER: Sahil S. Thakkar (Crypto & Identity Lead)

"Thank you, Subeer. In legacy architectures, service identity is frequently passed in HTTP headers, such as X-User-ID or X-Service-Name. Any junior penetration tester knows that HTTP headers can be easily spoofed using curl or a proxy.
In ZetaShield, we adopted the industry-standard **SPIFFE (Secure Production Identity Framework for Everyone)** standard, the exact same identity model used by Istio and HashiCorp Consul. Service identity is cryptographically bound into the X.509 certificate's **Subject Alternative Name (SAN)** URI extension—for example, spiffe://zetashield.local/ns/default/sa/frontend-service.
Why is this unforgeable? Because during the TLS handshake, our Go gateway extracts the identity directly from r.TLS.PeerCertificates[0].URIs. To forge an identity, an attacker would need our private Root CA key to sign their certificate.
In our adversarial tests, an attacker creates a certificate with the exact legitimate SPIFFE identity URI, but signs it themselves. The gateway drops them at the handshake because the issuer is untrusted. Subeer will now walk through our policy and hot-reload mechanisms."

MONEY LINE: "The attacker forged the exact identity on the badge, but trust lives in the issuer, not the claim."

SLIDE 6: Deny-by-Default & Hot-Reload (2.0 min)

SPEAKER: Subeer Srivastava (Policy Architect)

"Thank you, Sahil. Our access control rules are defined in a clean, human-readable policy.yaml file. It consists of two sections: route prefix mappings and access rules.
Notice the zero-trust micro-segmentation in action: frontend-service is explicitly allowed to call GET /orders/ and POST /payments/. orders-service is allowed to query its own domain, /orders/. Crucially, there is **NO RULE** permitting orders-service to access /payments/. Therefore, if an attacker compromises the orders service, lateral movement into the payments API is strictly blocked with a 403.
Furthermore, in V2.0 we engineered **zero-downtime Policy Hot-Reload**. A dedicated goroutine in our gateway checks the modification timestamp of policy.yaml every 1 second using standard-library os.Stat. When an administrator edits the policy, the gateway detects it, acquires a write lock on our sync.RWMutex, and atomically reloads the rules in memory in under 1 second. Active connections are never dropped, and no container restart is required. Next, let us review our 7-test verification matrix."

MONEY LINE: "The security perimeter changes in seconds without zero downtime or dropped TCP sessions."

SLIDE 7: The 7-Test Adversarial Suite (2.0 min)

SPEAKER: Subeer Srivastava (Policy Architect)

"To prove our architecture works beyond theoretical claims, we engineered a deterministic 7-test adversarial test client. Every test maps to a concrete real-world threat scenario:
Test 1 verifies Authorized Access: frontend-service calls /orders/list with its valid certificate → **200 OK**.
Test 2 verifies Transport Security: an anonymous client connects without a certificate → **TLS Handshake Rejected**.
Test 3 verifies Deny-by-Default: frontend-service probes /admin/delete where no rule exists → **403 Forbidden**.
Test 4 is our Micro-Segmentation flagship: orders-service, fully authenticated, attempts to query /payments/balance. Because no rule exists, it receives **403 Forbidden**—lateral movement is halted.
Test 5 verifies Own-Domain Access: payments-service accesses its own balance → **200 OK**.
Test 6 verifies Cryptographic Authenticity: an attacker presenting a forged certificate with the legitimate SPIFFE identity is rejected at the TLS handshake.
Test 7 verifies Fail-Closed Availability: a request to an offline inventory backend returns **503 Service Unavailable** without failing open.
The deterministic expected sequence is: **200, TLS_REJECT, 403, 403, 200, TLS_REJECT, 503**. I now turn over to Karthik to drive our live demonstration."

MONEY LINE: "Fully authenticated, still denied — authentication is not authorization."

SLIDE 8: Live Demonstration Execution (6.0 min)

DRIVER: Karthik (Lead) | NARRATORS: Sameep (Isolation) & Subeer (Hot-Reload)

(See Part 3: Live Demo Runbook for the full beat-by-beat terminal execution transcript).

SLIDE 9: Docker Network Isolation: Solving Deployability (1.5 min)

SPEAKER: Polisetti Hari Sri Raj Karthik (Lead)

"Thank you. In Review 1, the committee asked: *'Does this only work on localhost under narrow conditions? How is this deployable?'*
This slide presents our architectural answer. In local development mode, we enforced isolation by binding backends exclusively to 127.0.0.1 loopback. In V2.0, we adapted the exact same isolation invariant to containerized deployments using **Docker bridge network namespaces**.
Our Compose file defines a private bridge network called zeta-net. The Orders and Payments containers have **no port bindings whatsoever**. The Gateway is the sole container with a published port: 8443:8443. Because Docker provides internal DNS resolution, the gateway proxies traffic to http://orders:9091 and http://payments:9092 over the bridge. The host machine cannot route to these services directly. This is not a hack—this is the exact network namespace isolation pattern used in production Kubernetes clusters."

MONEY LINE: "The isolation invariant survives the platform shift: from loopback binding to container network namespaces."

SLIDE 10: Sub-Millisecond Enforcement Benchmark (1.5 min)

SPEAKER: Polisetti Hari Sri Raj Karthik (Lead)

"Another major critique from Review 1 was: *'Too many layers introduces too much latency.'*
Rather than guessing, we measured. We engineered a dedicated benchmarking tool in cmd/loadtest that executes 500 sequential requests comparing direct calls against calls through our full Zero-Trust gateway on the same hardware.
Here are the empirical results: direct requests to the backend average 0.75 milliseconds median latency. Requests traversing the full Zero-Trust gateway—which includes the complete mTLS handshake, cryptographic SAN extraction, in-memory policy lookup, reverse proxy forwarding, and synchronous JSONL audit disk write—average 1.35 milliseconds.
The total median enforcement overhead is **only ~0.60 milliseconds**, and under 1.2 milliseconds at the 99th percentile. In production microservices where network calls take 20 to 100 milliseconds, a 0.6ms overhead represents an imperceptible 1 to 3 percent cost. Now, Sameep will discuss our unit testing and CI pipeline."

MONEY LINE: "Full mTLS and L7 policy enforcement costs only 0.6 milliseconds — zero trust does not mean zero performance."

SLIDE 11: Engineering Rigor: 19 Unit Tests & CI (1.5 min)

SPEAKER: Sameep Upadhyay (Verification Engineer)

"Thank you, Karthik. To ensure production stability and prevent regression errors, we implemented 19 comprehensive, table-driven unit tests in Go.
These cover two distinct categories: **14 Policy Tests** verifying authorized flows, own-domain access, method constraints like blocking POST on GET-only routes, root probes, admin probes, empty identities, and cross-service denials. And **5 Routing Tests** verifying longest-prefix matching, dead routes, and unknown endpoints.
Furthermore, this is not just tested on our laptops. Every single git push to our GitHub repository triggers an automated **GitHub Actions Continuous Integration pipeline**. The runner spins up an Ubuntu environment, runs go vet, builds the static binary, and executes all 19 unit tests. Our repository has maintained a 100% green build badge across our entire development cycle. Shaivyaa will now walk us through our design evolution."

MONEY LINE: "19 automated unit tests gating every commit in CI — engineering rigor, not just a working prototype."

SLIDE 12: Three-Version Design Evolution (1.5 min)

SPEAKER: Shaivyaa Sharma (Security Analyst)

"Thank you, Sameep. Great engineering is defined by how effectively it responds to peer and supervisor critique. This slide illustrates our design evolution across three distinct versions:
In **V0.1**, presented at Review 1, we implemented a 4-hop sidecar prototype. The feedback was direct: too complex, high latency, single backend.
In response, we engineered **V1.0**: we consolidated the architecture into a 2-hop gateway, added mutual TLS with SPIFFE identities, built a multi-backend service fleet, added 16 unit tests, and proved the 0.6ms latency benchmark.
Finally, for Review 2, we built **V2.0**: complete Docker containerization, network namespace isolation, tests 6 and 7 for forged certs and fail-closed outages, 19 unit tests, and real-time policy hot-reload.
Every single critique from Review 1 has been addressed with working code and empirical data. Next is our Phase 2 roadmap."

MONEY LINE: "We converted every criticism from Review 1 into a verifiable, working engineering feature."

SLIDE 13: Phase 2 Technical Roadmap (1.5 min)

SPEAKER: Shaivyaa Sharma (Security Analyst)

"With our core data-plane enforcement fully proven in Phase 1, our Phase 2 roadmap expands into management, scaling, and validation:
First is the React Administrative Console: We will construct a real-time web UI consuming the audit stream via WebSockets, featuring a live topology map showing service connections and blocked lateral movement attempts, alongside an interactive policy editor.
Second is Cloud Deployment: We will deploy the container mesh to a public cloud VPS to demonstrate internet-facing zero-trust ingress.
Third is Automated Red-Team Simulation: We will build a dedicated attacker container inside the mesh that continuously fuzzes and probes the gateway.
Fourth is eBPF Kernel Acceleration: Exploring kernel-level Layer 4 packet filtering to complement our Layer 7 proxy. Karthik will now summarize our outcomes."

SLIDE 14: Project Outcomes & Industry Value (1.0 min)

SPEAKER: Polisetti Hari Sri Raj Karthik (Lead)

"In conclusion, ZetaShield achieves three fundamental milestones: First, it replaces fragile, IP-based perimeter trust with **cryptographic, identity-based micro-segmentation**. Second, it delivers provable, fail-closed security guarantees at a negligible **0.6ms latency overhead**. Third, it provides organizations with a deployable, lightweight internal firewall for server-to-server traffic without the crushing operational complexity of an enterprise service mesh."

MONEY LINE: "A deployable, identity-based firewall proxy for traffic between your own servers."

SLIDE 15: Team Contributions Breakdown (1.0 min)

SPEAKER: Sameep Upadhyay (Verification Engineer)

"This project represents an intensive, balanced collaborative effort across the entire software development lifecycle: Karthik led core gateway engineering, mTLS logic, Docker mesh deployment, CI/CD, and benchmarking. Subeer engineered the policy schema, deny-by-default logic, PBAC evaluation, and hot-reload engine. Sahil directed the PKI architecture, X.509 extensions, and SPIFFE identity model. Shaivyaa conducted the threat modeling, literature investigation, and Phase 2 dashboard architecture. And I designed the 19-test unit suite, adversarial client scenarios, and network isolation verification."

SLIDE 16: Summary & Technical Q&A; Defense (1.0 min)

SPEAKER: Polisetti Hari Sri Raj Karthik (Lead)

"To summarize: ZetaShield V2.0 is **Code Complete, Fully Containerized, Empirically Benchmarked, and Strictly Fail-Closed**. Our source code, CI pipelines, and reproduction guides are public on GitHub under release tag v1.0-phase1.
We thank our supervisor Dr. Sajjad Ahmed for his invaluable guidance. We are now open and ready for your questions."

Part 3: The Live Demo Runbook (6-Minute Protocol)

Follow this beat-by-beat protocol with military precision. **Never debug or recompile code during the presentation.** If a terminal hangs, transition immediately to the contingency protocol.

Beat / Step	Terminal & Exact Command	Spoken Narration & Action	Expected Terminal Output

BEAT 1
Mesh Launch
(45 sec)
	Terminal 1 (Root):
	docker compose up --build -d
	
	docker compose ps
		Karthik: *'We launch the entire Zero-Trust mesh with one command. Notice the container status: the Gateway maps port 8443 to the host. Orders and Payments have ZERO published host ports.'*	`zeta-gateway: Up (0.0.0.0:8443->8443)`
			`zeta-orders: Up (no ports)`
			`zeta-payments: Up (no ports)`

BEAT 2 Isolation Proof *(45 sec)*	**Terminal 2 (Host):** curl http://127.0.0.1:9091/orders/list curl http://127.0.0.1:9092/payments/balance	**Sameep:** *'Here is our proof of network namespace isolation. An attacker on the host tries to curl the Orders backend directly. It fails with Connection Refused. The only path is through the mTLS gateway.'*	`curl: (7) Failed to connect to 127.0.0.1 port 9091: Connection refused`

BEAT 3
7-Test Suite
(90 sec)
	Terminal 2 (Host):
	cd mock-apps
	python client.py
		Karthik: Runs script.
		Subeer: Narrates tests 1–4.
		Sahil: Narrates test 6 (forged SAN rejection).
		Karthik: Concludes test 7 (fail-closed 503).
			`[TEST 1] Authorized → 200 OK`
			`[TEST 2] No Cert → TLS_REJECT`
			`[TEST 3] Admin Probe → 403 Forbidden`
			`[TEST 4] Lateral Move → 403 Forbidden`
			`[TEST 5] Own Domain → 200 OK`
			`[TEST 6] Forged Cert → TLS_REJECT`
			`[TEST 7] Fail-Closed → 503`
			`Sequence: 200, TLS_REJECT, 403, 403, 200, TLS_REJECT, 503`

BEAT 4
Hot-Reload Flex
(90 sec)
	Terminal 3 (Logs):
	docker compose logs -f gateway
	
	VS Code: Comment out payments rule in deploy/policy.docker.yaml
	
	Terminal 2: python client.py
		Subeer: *'Watch the gateway logs. We edit policy.docker.yaml to revoke payments-service access. Within one second, the gateway logs [policy] hot-reloaded without container restart. We re-run the client: Test 5 instantly flips from 200 OK to 403 Forbidden! Access revoked live.'*
		(Re-uncomment rule after demo).
			`zeta-gateway | [policy] hot-reloaded /etc/zetashield/policy.yaml`
			
			`[TEST 5] Payments → 403 Forbidden (LIVE REVOCATION)`

BEAT 5 Audit Trail *(45 sec)*	**Terminal 2 (Host):** tail -n 7 deploy/logs/audit.log.jsonl	**Sameep:** *'Every single request produces an immutable structured JSONL audit entry. Notice the entries: decisions ALLOW, DENY, and ERROR with complete cryptographic identity and reason.'*	`{"timestamp":"...","decision":"DENY","identity":"spiffe://../orders-service","reason":"no matching ALLOW rule"}`
BEAT 6 Disaster Protocol *(Fallback)*	**VLC Media Player:** ZetaShield_V2_Demo_Backup.mp4 (Pre-loaded, paused)	**Karthik (Rule):** If Docker fails or host hangs, switch to video immediately. **Never troubleshoot code in front of reviewers.**	*'As an engineering redundancy, we recorded the identical containerized demo run in full HD...'*

Part 4: Viva Q&A; Defense Bank (20 Questions)

Questions are organized by individual territory. Answer directly, confidently, and cite working implementation artifacts. Avoid rambling—give the technical punchline in the first sentence.

Territory 1: Architecture, Dangerous Trap Questions & Scalability (Karthik)

Q1 (Trap): Why did you drop the separate FastAPI Control Plane from your original V0.1 design?
Karthik: "We consolidated based directly on Review 1 feedback. In V0.1, calling an external control plane added a fourth network hop, inflating median latency to ~2.5ms. In modern zero-trust design (like Istio Ambient mode), in-process policy evaluation is preferred for performance. By embedding the policy engine in Go with sync.RWMutex and file hot-reload, we slashed median overhead to 0.6ms while preserving dynamic policy updates. A decoupled control plane is reserved for Phase 2 when managing multi-cluster fleets."

Q2 (Trap): Isn't a consolidated gateway a Single Point of Failure (SPOF)?
Karthik: "In our Phase 1 prototype, yes, it runs as a single container. However, architecturally, our gateway is completely stateless—it maintains no session state or distributed locks. In production, you scale it horizontally behind a Layer-4 cloud load balancer (like AWS NLB or HAProxy), with multiple gateway instances sharing the same read-only policy volume or control plane. If one crashes, traffic instantly shifts to healthy nodes."

Q3 (Trap): 500 requests is not real benchmarking. How do you know this scales to 50,000 req/sec?
Karthik: "The 500-request benchmark in cmd/loadtest was specifically designed to isolate per-request enforcement overhead on identical hardware without network jitter. Go's net/http stack natively allocates an ultra-lightweight 2KB goroutine per connection. Because our policy check is a sub-microsecond in-memory hash map lookup under an RWMutex read lock, throughput is purely bounded by CPU TLS cryptography. For Phase 2, we have scheduled concurrent wrk stress-testing."

Q4: Why did you build this in Go instead of Python or Rust?
Karthik: "Go was chosen for three architectural reasons: First, its concurrency model uses lightweight goroutines rather than Python's Global Interpreter Lock (GIL). Second, Go's standard library net/http/httputil and crypto/tls packages are battle-tested production stacks used by Cloudflare and Docker. Third, Go compiles to a static, self-contained binary under 15MB, eliminating runtime dependency vulnerabilities in our Alpine containers."

Q5: How does this differ from commercial solutions like Istio or Linkerd?
Karthik: "Istio requires a heavy Kubernetes control plane (istiod) and injects Envoy sidecars consuming 50MB+ of RAM per container. ZetaShield provides the exact same core NIST SP 800-207A guarantees—mTLS, SPIFFE identity, deny-by-default policy, fail-closed routing—in a standalone, 15MB binary runnable on any laptop with single-command Docker Compose simplicity."

Q6: What happens if an attacker compromises the Docker host itself?
Karthik: "Zero Trust operates on the principle of defense-in-depth within the application plane. If the underlying host kernel is compromised, host-level security (SELinux/AppArmor) is the relevant boundary. However, within the container plane, ZetaShield guarantees that compromised workloads cannot move laterally across the bridge network to reach other microservices without cryptographic authorization."

Territory 2: Policy Engine, Hot-Reload & Concurrency (Subeer)

Q7: What happens if an administrator saves a malformed or corrupted YAML policy during hot-reload?
Subeer: "The gateway strictly fails closed. In loadPolicy(), we parse the YAML using gopkg.in/yaml.v3. If parsing fails, the error is logged to stderr, the reload aborts, and the gateway retains the previous valid in-memory policy. At no point does a corrupted file zero out the rules or cause a panic."

Q8: How do you prevent race conditions when reloading policy while requests are executing?
Subeer: "We synchronize state using Go's sync.RWMutex. When requests are being authorized in checkPolicy(), they acquire a read lock (RLock()), allowing thousands of concurrent evaluations. When startPolicyWatcher() detects a file modification, it acquires an exclusive write lock (Lock()) on policyMu. New requests pause for a fraction of a microsecond while the struct pointer updates, guaranteeing zero race conditions and zero half-read state."

Q9: Why did you use a 1-second time.Ticker with os.Stat instead of Linux fsnotify?
Subeer: "fsnotify relies on inotify syscalls, which are notoriously flaky across Docker volume mounts on Windows and macOS. When Docker mounts a host file, inode events are frequently dropped. A 1-second polling ticker inspecting os.Stat().ModTime() is 100% portable, standard-library native, consumes negligible CPU, and guarantees reliable detection across all platforms."

Q10: What if an attacker tries path traversal, like /orders/../admin?
Subeer: "Go's net/http standard library automatically canonicalizes request paths using path.Clean before passing the request to our handler. Any ../ sequence is resolved before checkPolicy() evaluates strings.HasPrefix. Furthermore, our deny-by-default logic blocks any unmatched path."

Q11: Why is deny-by-default structurally safer than permit-by-default?
Subeer: "In a permit-by-default system, a missing rule or typo opens a security hole. In ZetaShield, the safe state is the default state: if an identity has no explicit rule for a path and method, checkPolicy() returns false. An empty policy file results in 100% rejection, which is the foundational rule of Zero Trust."

Territory 3: Cryptography, PKI & SPIFFE Identities (Sahil)

Q12: What happens if your private Root CA key is compromised?
Sahil: "If the Root CA key is compromised, an attacker can issue valid certificates. In our Phase 1 prototype, CA keys are generated locally. In a production environment, the Root CA resides in a Hardware Security Module (HSM) or an enterprise PKI like HashiCorp Vault. In Phase 2, we plan to integrate SPIRE, which issues short-lived 1-hour certificates with automated workload attestation, drastically reducing the blast radius."

Q13: Why did you embed the SPIFFE URI in the Subject Alternative Name (SAN) instead of the Common Name (CN)?
Sahil: "RFC 5280 and modern TLS standards officially deprecate the Common Name (CN) field. Furthermore, SPIFFE RFC specifications mandate that workload identity must be carried in the Subject Alternative Name (SAN) URI extension. This cleanly decouples the transport hostname from the cryptographic service identity."

Q14: Why do service certificates have a 30-day validity period?
Sahil: "Long-lived certificates violate zero-trust hygiene because compromised keys remain valid indefinitely. 30 days was chosen as a practical developer validity. In production with automated rotation, certificates should expire in 24 hours or less."

Q15: How did you fix the 'IP address mismatch' error during proxy certificate generation?
Sahil: "When clients connect over HTTPS to 127.0.0.1, the TLS stack verifies that the server certificate SAN matches the connect address. Initially our proxy certificate only contained the SPIFFE URI. We updated issue_cert.sh so that when the service is 'proxy', it appends IP:127.0.0.1 and DNS:localhost to san.cnf, enabling standard hostname verification without disabling client-side security checks."

Territory 4: Testing, Verification & Roadmap (Sameep & Shaivyaa)

Q16: How do you prove backends cannot talk to each other without going through the gateway?
Sameep: "In local mode, services bind to 127.0.0.1. In Docker mode, services reside in an isolated bridge network with no host ports published. If orders-service attempts to query payments-api, it cannot route directly because network policies restrict inter-container traffic. In Test 4, when orders-service calls the gateway for payments, the gateway returns 403 Forbidden."

Q17: Why do you have 19 unit tests? What do they cover?
Sameep: "Our 19 tests in policy_test.go provide 100% branch coverage of our authorization logic: 14 policy tests cover authorized paths, method enforcement (POST vs GET), root probes, admin probes, empty identities, and cross-service denials. The 5 routing tests verify longest-prefix route resolution and dead routes."

Q18: Why does Test 7 use 'dead-host:9099' in Docker instead of an IP address?
Sameep: "In local mode, port 9099 is offline, triggering a TCP connection refused error. In Docker, dead-host is an unresolvable hostname, triggering a DNS resolution failure. Both test cases prove that when an upstream backend fails, our custom ErrorHandler intercepts the failure and returns a controlled HTTP 503 rather than failing open."

Q19: How will Phase 2 improve on Phase 1?
Shaivyaa: "Phase 1 proved data-plane enforcement and sub-millisecond overhead. Phase 2 focuses on operational observability: a React web console with a live WebSocket feed of audit.log.jsonl, automated red-team probing, and eBPF kernel acceleration for Layer 4 traffic."

Q20: Can ZetaShield be deployed in an existing Kubernetes cluster?
Shaivyaa: "Yes. ZetaShield can be deployed as an Ingress Controller or API Gateway using Kubernetes DaemonSets or Deployments, with backends isolated via K8s NetworkPolicies."

THE 3 GOLDEN SENTENCES (MUST BE MEMORIZED BY ALL MEMBERS):

1. *'Zero Trust means trust is never implicit; every single request must be cryptographically authenticated, authorized by policy, and audited.'*
2. *'Authentication is not authorization — presenting a valid certificate gets you through the TLS door, but policy determines if you can touch the data.'*
3. *'Failure is never permission — if a certificate is missing, a rule is absent, or a backend is down, ZetaShield strictly fails closed.'*

Part 5: Team Briefing (Copy-Paste Group Message)

Copy and paste the exact message block below into your team WhatsApp or Discord group chat 24 hours before Review 2:

TEAM BRIEFING — CAPSTONE REVIEW 2 FINAL BATTLE STATIONS

Team, here is our locked operational plan for tomorrow's Review 2 presentation:

1. SPEAKING ORDER & TIMINGS (Strict 24 Minutes):
• Slides 1–3: Karthik (Lead) — Opening, Problem & Architecture (4.5 min)
• Slide 4: Subeer — Three Checkpoints & Enforcement Pipeline (2.0 min)
• Slide 5: Sahil — Cryptographic Identity & SPIFFE SANs (2.0 min)
• Slides 6–7: Subeer — Deny-by-Default Policy, Hot-Reload & 7-Test Matrix (4.0 min)
• Slide 8: LIVE DEMO (Karthik drives, Sameep/Subeer narrate isolation & hot-reload) (6.0 min)
• Slides 9–10: Karthik — Docker Network Isolation & 0.6ms Benchmark (3.0 min)
• Slide 11: Sameep — 19 Unit Tests & GitHub Actions CI (1.5 min)
• Slides 12–13: Shaivyaa — 3-Version Evolution & Phase 2 Roadmap (3.0 min)
• Slide 14: Karthik — Project Outcomes (1.0 min)
• Slide 15: Sameep — Team Contributions Table (1.0 min)
• Slide 16: Karthik — Summary & Transition to Q&A; (1.0 min)

2. THE GOLDEN HANDOFF PHRASE:
Always pass the mic with authority: *'I will now hand over to [Name] to walk through [Topic].'*

3. THE RESCUE PROTOCOL (IF SOMEONE FREEZES):
If a member freezes for more than 4 seconds on slide or viva, Karthik (or territory lead) steps in smoothly using this exact phrase:
'To build on what [Name] stated regarding the underlying implementation...'
Never say 'he forgot' or 'let me correct that.' Keep the team facade unified.

4. DEMO DISASTER PROTOCOL:
If Docker hangs or Windows network glitches, **DO NOT DEBUG LIVE**. Karthik will immediately switch to our pre-recorded MP4 backup video in VLC.

5. DRESS CODE & SETTINGS:
Formal attire. Join 15 minutes early. Mute all notifications. Have water nearby. Let's execute!

Part 6: Rehearsal Protocol & Practice Schedule

To guarantee an A-grade evaluation, the team must complete two structured rehearsal runs prior to stepping into the review room.

Rehearsal Drill	Target Duration	Key Objective & Execution Rules
Drill 1: Full Dress Rehearsal (Evening Before)	60 Minutes	• Run through all 16 slides with timer running. • **Rule: NO STOPPING FOR MISTAKES.** Push through hesitations to test recovery. • Execute the live demo twice, including the live policy hot-reload flex. • Conduct a 15-minute cross-examination using the Viva Q&A; Bank.
Drill 2: Cold-Start Dry Run (Morning of Review)	30 Minutes	• Perform a cold-start live demo from clean terminal state. • Verify Docker containers build and start without network lag. • Run client.py and confirm the 7-test sequence is crisp. • Rapid-fire recitation of the 3 Golden Sentences.

Part 7: Day-of Execution Checklist (T-Minus 30 Minutes)

Execute every item on this checklist 30 minutes before your scheduled presentation slot.

Phase / Time	Action Item & Verification Task	Owner
T-30 min Environment	• Start Docker Desktop and verify engine is running (docker info). • Pre-warm build cache: run docker compose build so images are cached. • Clear existing audit logs: type nul > deploy/logs/audit.log.jsonl.	**Karthik**

T-20 min Workspace Layout	• Open 3 distinct terminal windows positioned across screen: - Terminal 1 (Top-Left): docker compose up / status - Terminal 2 (Bottom-Left): mock-apps for running client.py & curl - Terminal 3 (Right): docker compose logs -f gateway for hot-reload logs • Set terminal font size to 16pt bold (Monospace) for panel visibility.	**Karthik / Sameep**
T-15 min Redundancy	• Open VLC Media Player with ZetaShield_V2_Demo_Backup.mp4 loaded and paused. • Open PowerPoint / PDF presentation slides in full-screen presentation mode. • Open GitHub repository page in browser tab on commit f4a92b9.	**Karthik / Shaivyaa**
T-10 min Distraction Guard	• Enable Windows 'Focus Assist' / Do Not Disturb. • Close WhatsApp, Discord, Telegram, Slack, and email desktop clients. • Check microphone input levels and camera lighting. • Ensure laptop is plugged into AC power (High Performance battery profile).	**All Members**
T-5 min Mental Focus	• Glass of water at desk. • Deep breath. Remember the core message: *We built a working Layer-7 Zero-Trust gateway in Go that proves sub-millisecond enforcement.* • Stand tall, speak clearly, and own the room.	**All Members**